proc_monitor

Linux System Monitoring Tool

Using the `/proc` Filesystem

Technical Documentation & Test Report

S.No	Roll Number	Name
1	23CS10013	Chintada Yesheeth Sree Narayana
2	23CS10017	Dasari Veera Venkata Abhinav Teja
3	23CS10052	Hadwik Payidiparthy

Advanced Operating Systems — Autumn 2026–2027

Assignment 1

August 14, 2026

Contents

1 Overview

`proc_monitor` is a command-line utility that reads directly from the `/proc` virtual filesystem to report CPU, memory, and process information. It does not invoke any external tools such as `ps`, `top`, or `free`.

The tool operates in two modes:

- **Static mode** — a single snapshot of all metrics, then exits.
- **Live mode** — auto-refreshing display every T seconds until the user presses `Ctrl+C`.

2 File Structure

```
proc_monitor/
 Makefile
 README.md
 src/
     main.cpp          # Entry point  decides static vs live mode
     cpu_info.hpp
     cpu_info.cpp      # Task 1  CPU model and logical processor count
     mem_info.hpp
     mem_info.cpp      # Task 1  Memory usage statistics
     stats_info.hpp
     stats_info.cpp    # Task 2  CPU utilization, uptime, load averages
     proc_info.hpp
     proc_info.cpp     # Task 3 & 4  Process listing and top consumers
     display.hpp
     display.cpp       # Formatted output for all tasks
```

Each source file maps to a distinct `/proc` domain, making it easy to isolate bugs and extend individual modules independently.

3 Design Choices

3.1 Two Separate 1-Second Sampling Windows

The tool performs two independent 1-second sleeps:

- One in `stats_info` for overall CPU usage (reads `/proc/stat` twice).
- One in `proc_info` for per-process CPU usage (reads `/proc/[pid]/stat` twice).

This means the minimum meaningful live mode interval is $T > 2$ seconds. For $T \leq 2$, the tool exits with a warning:

```
Warning: Interval less than or equal to 2 seconds may not provide
accurate CPU usage data.
```

Testing confirmed: $T = 2\,\text{s}$ fails; $T \geq 3\,\text{s}$ works correctly.

3.2 Per-Process CPU Usage Formula

Per-process CPU usage is computed as:

$$\text{CPU\%} = \frac{\Delta_{\text{ticks}}}{\text{ticks\_per\_second}} \times 100$$

where $\Delta_{\text{ticks}} = (\text{utime}_2 + \text{stime}_2) - (\text{utime}_1 + \text{stime}_1)$ over a 1-second interval. `ticks_per_second` is read from `sysconf(_SC_CLK_TCK)`, with a fallback of 100 Hz.

This differs from the overall CPU usage formula, which uses system-wide `/proc/stat` deltas as the denominator:

$$\text{CPU\%}_{\text{system}} = \left(1 - \frac{\Delta_{\text{idle}}}{\Delta_{\text{total}}}\right) \times 100$$

3.3 Process Name Truncation

Names are read from `/proc/[pid]/status`, which truncates at 15 characters due to a kernel limitation. Names like `init-systemd(Ub` appearing cut off in the output is expected and correct behaviour.

3.4 Kernel Threads Show VmSize = 0

Kernel threads (e.g. `kthreadd`) have no user-space memory mapping, so `/proc/[pid]/status` contains no `Vm*` fields. These default to 0 in the output.

3.5 Screen Clearing in Live Mode

Live mode uses `system("clear")` for reliable terminal compatibility across different WSL terminal emulators.

3.6 Graceful Handling of Process Death Mid-Read

Processes can exit between directory listing and status file read. The tool silently skips such processes via the `file.is_open()` check in `read_process_status()`, ensuring no crash or error output.

4 Test Cases

4.1 Test 1 — Static Mode Basic Run

Command	`./proc_monitor`
Expected	Single snapshot of all 4 sections, then exit
Verdict	**PASS**

Output:

```
============================================================
 PROC MONITOR - SYSTEM INFORMATION
============================================================
[1] CPU & MEMORY INFORMATION
------------------------------------------------------------
CPU Model        : AMD Ryzen 7 5800HS with Radeon Graphics
Logical CPUs     : 16
Total Memory     : 7.46 GB
Available Memory : 6.82 GB
Free Memory      : 6.59 GB
Memory Usage     : 8.6%
------------------------------------------------------------
[2] CPU UTILIZATION & SYSTEM STATISTICS
------------------------------------------------------------
```

```
CPU Usage         : 0.1%
System Uptime     : 0 Days 0 Hours 6 Minutes
Load Average
  1 min : 0.14
  5 min : 0.11
 15 min : 0.04
------------------------------------------------------------
[3] RUNNING PROCESSES
------------------------------------------------------------
PID     PPID    STATE   THREADS   VmSize(MB)    NAME
1       0       S       1         21            systemd
2       1       S       2         2             init-systemd(Ub
...
656     331     R       1         6             proc_monitor
------------------------------------------------------------
[4] TOP 5 MEMORY CONSUMERS
------------------------------------------------------------
Rank    PID     Memory(MB)    Process
1       215     29            postgres
2       191     21            unattended-upgr
3       159     17            wsl-pro-service
4       55      15            systemd-journal
5       1       12            systemd
------------------------------------------------------------
TOP 5 CPU CONSUMERS
------------------------------------------------------------
Rank    PID     CPU(%)    Process
1       242     0.0       postgres
2       656     0.0       proc_monitor
...
============================================================
```

Observations:

- CPU model and logical CPU count (16) correctly identified from `/proc/cpuinfo`.
- Memory total 7.46 GB, usage 8.6% — consistent with an idle WSL environment.
- System uptime correctly parsed from `/proc/uptime` and formatted as Days/Hours/Minutes.
- 32 processes listed with correct PID, PPID, state, threads, and VmSize.
- `proc_monitor` itself visible in state `R` (Running), confirming the tool correctly sees itself as a running process.
- `postgres` correctly ranked first in memory with 29 MB RSS.
- All CPU consumers show 0.0% on the idle system — correct, no active workload.

4.2 Test 2 — Live Mode with Active CPU Workload

Command	`./proc_monitor -T 10`
Workload	`while true; do :; done` (separate terminal)
Expected	CPU and process list update each refresh; busy process appears in top CPU consumers
Verdict	**PASS**

Refresh 1 (before workload visible in sample window):

```
CPU Usage        : 0.1%
System Uptime    : 0 Days 0 Hours 4 Minutes
Load Average
  1 min : 0.18   5 min : 0.09   15 min : 0.03
...
TOP 5 CPU CONSUMERS
Rank    PID     CPU(%)    Process
1       242     0.0       postgres
```

Refresh 2 (workload active during sample window):

```
CPU Usage        : 6.2%
System Uptime    : 0 Days 0 Hours 5 Minutes
Load Average
  1 min : 0.21   5 min : 0.10   15 min : 0.03
...
473     466     R       1         6             bash      <- state R
...
616-622  99     S       1         23            (udev-worker) <- new
...
TOP 5 CPU CONSUMERS
Rank    PID     CPU(%)    Process
1       473     100.0     bash
```

Observations:

- CPU usage jumped from 0.1% to 6.2% — workload correctly detected via `/proc/stat` delta.
- Load average (1 min) rose from 0.18 to 0.21, reflecting the growing workload.
- `bash` (PID 473) state changed from `S` (sleeping) to `R` (running).
- `bash` (PID 473) appeared at the top of CPU consumers with 100.0%.
- 6 new `udev-worker` processes (PIDs 616–622) appeared dynamically without restarting the monitor, confirming live `/proc` reading works correctly.
- Process list grew from 31 to 37 entries between refreshes.

4.3 Test 3 — Minimum Interval Enforcement

Command	`./proc_monitor -T 1`
Expected	Warning message and clean exit
Verdict	**PASS**

Output:

```
Warning: Interval less than or equal to 2 seconds may not provide
accurate CPU usage data.
```

$T = 2$ triggers the same warning. $T = 3$ is the minimum working interval, since the tool uses two sequential 1-second sampling windows internally.

4.4 Test 4 — Process Count Consistency

Commands	`ls /proc | grep -c "^[0-9]"` → 33
	`./proc_monitor > out.txt`
	`awk '/RUNNING PROCESSES/,/TOP 5 MEMORY/' out.txt`
	`| grep -c "^[0-9]"` → 32

/proc count	33
proc_monitor count	32
Difference	1
Verdict	**PASS**

Note: `/proc` is read first, then `proc_monitor` runs. In the time between the two, one process may have exited or a new transient process appeared. This is a natural race condition when reading a live virtual filesystem sequentially. The tool handles this gracefully by silently skipping any process whose status file disappears mid-read.

5 What Gets Reported

Section	Source	Description
CPU Info	`/proc/cpuinfo`	Model name, logical CPU count
Memory Info	`/proc/meminfo`	Total, free, available memory and usage %
CPU Utilization	`/proc/stat`	CPU usage % over a 1-second interval
System Uptime	`/proc/uptime`	Days, hours, minutes since boot
Load Averages	`/proc/loadavg`	1, 5, and 15 minute load averages
Process List	`/proc/[pid]/status`	PID, PPID, state, threads, VmSize for all processes
Top 5 Memory	`/proc/[pid]/status`	Top 5 processes by RSS (actual RAM in use)
Top 5 CPU	`/proc/[pid]/stat`	Top 5 processes by CPU usage over 1-second interval